

ADD y Trade-offs

Attribute Driven Design (5 pasos) · árbol de utilidad · trade-offs canónicos · CAP

Ingeniería de Software II · ITBA · 2026-1C · Repaso conceptual

RESUMEN

ADD (Attribute Driven Design) construye una arquitectura paso a paso haciendo **explícita la trazabilidad** entre cada descomposición y el atributo de calidad que la justifica. Su combustible son los **drivers** (functional + QA + constraints), priorizados con un **árbol de utilidad**. Y como ningún atributo mejora en el vacío, cada decisión es un **trade-off** — CAP es el caso canónico.

§1 Attribute Driven Design (ADD)

DEFINICIÓN

ADD: método para **construir** una arquitectura (no sólo describirla) que sustituye el "partimos en 3 capas porque siempre se hizo así" por "partimos en 3 capas porque X, Y, Z". Cada nodo del árbol estructural lleva un "*porque...*". Propuesto por Bass, Clements, Kazman (SEI/CMU) (source: attribute-driven-design.md).

Drivers arquitectónicos (los inputs)

ADD opera sobre tres tipos de input; los tres juntos son los **architectural drivers**. Todo lo demás es diseño de detalle (source: attribute-driven-design.md):

1. **Requisitos funcionales críticos** — los casos de uso que no pueden fallar.
2. **Atributos de calidad priorizados** — con escenarios concretos, salidos de un **árbol de utilidad** (ver §2).
3. **Restricciones (constraints)** — técnicas (stack legacy), de negocio (deadline, budget) o regulatorias.

Los 5 pasos

1. **Elegir módulo a descomponer.** Se arranca por el sistema completo (caja root); en iteraciones sucesivas, cada submódulo pasa a ser la caja a descomponer.
2. **Identificar drivers del módulo.** No todos los drivers del sistema aplican a cada módulo: ¿cuáles de los drivers totales recaen sobre *este* módulo?
3. **Elegir patrón/estilo que resuelva los drivers.** Aquí se toma la decisión arquitectónica. **Explicitar el trade-off:** todo patrón mejora algo a costa de algo — documentar el costo.
4. **Instanciar submódulos y asignar responsabilidades.** El patrón define roles (broker, client, worker...); se crean los submódulos, se asignan responsabilidades y se verifica que *juntos* sigan cubriendo los drivers del padre.

5. **Iterar.** Cada submódulo vuelve al paso 1. Se termina cuando la granularidad alcanza el nivel de "clase/componente codificable".

REGLA DE ORO

¿**Cuándo parar?** Cuando el equipo puede empezar a codear sin romper invariantes arquitectónicas. Más detalle que eso es diseño de bajo nivel, no arquitectura (source: attribute-driven-design.md).

CUÁNDO SÍ • VENTAJAS

vs. top-down clásico: el top-down parte de funcionalidad y descompone en módulos funcionales, produciendo árboles estructurales *sin* justificación. ADD parte de **atributos + funcionalidad**, descompone según qué atributo domina y obliga a trade-offs explícitos en cada nodo (source: attribute-driven-design.md).

CUÁNDO NO • CUIDADO

Cuándo NO aplicar: CRUD simples con atributos poco exigentes (overkill); spikes/prototipos (prima la velocidad de aprendizaje); equipos sin práctica con atributos de calidad (primero capacitar, no aplicar el método) (source: attribute-driven-design.md).

§2 Árbol de utilidad

DEFINICIÓN

Árbol de utilidad: estructura jerárquica que conecta el **valor abstracto** del sistema con **escenarios concretos y verificables**. Rompe la tentación de decir "queremos performance" y obliga a "queremos p95 < 100ms a 10k rps con 99.9% uptime durante un incidente de región". Se usa en [[ATAM]] y [[SAAM]] (source: arbol-de-utilidad.md).

Niveles

1. **Raíz** — utilidad general del sistema (nombre corto, ej. "Sistema de pagos").
2. **Nivel 1** — **Atributos de calidad** priorizados PERFORMANCE AVAILABILITY SECURITY
3. **Nivel 2** — **Features** dentro de cada atributo (latencia, throughput, uptime...).
4. **Hojas** — **Escenarios concretos**, anotados con (**importancia, dificultad**).

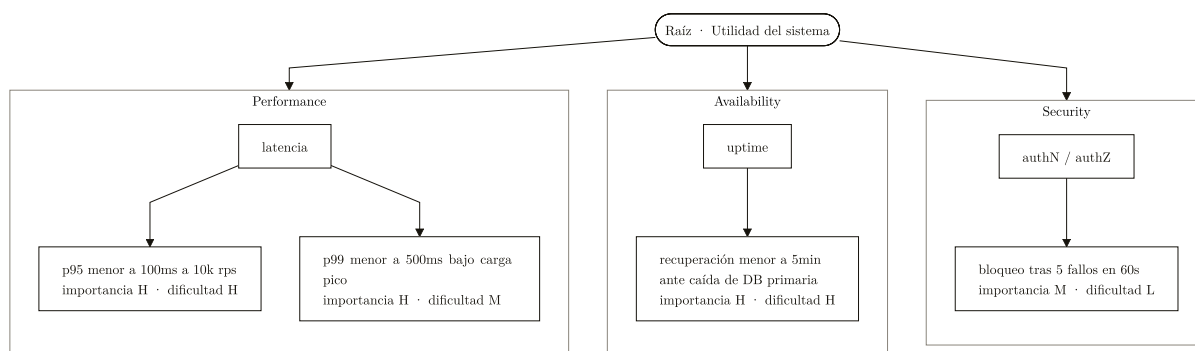


FIGURA 1. Árbol de utilidad: la raíz se ramifica en atributos de calidad (nivel 1), features (nivel 2) y hojas que son escenarios concretos anotados con (importancia, dificultad).

Anotación (importancia × dificultad)

Cada hoja se etiqueta con un par **H/M/L** (source: arbol-de-utilidad.md):

- **Importancia** (para el negocio): H = crítico · M = deseable · L = nice-to-have.
- **Dificultad** (técnica/arquitectónica): H = decisiones no-triviales · M = posible con buenas prácticas · L = trivial.

TABLA 1. Matriz de priorización (Imp × Dif)

Imp \ Dif	H	M	L
H	Focus principal (riesgo y esfuerzo altos)	Focus secundario	Implementar y verificar
M	Analizar trade-offs	Típico del backlog	Baseline
L	Reconsiderar si hace falta	Baja prioridad	Ignorar

Los escenarios **H/H** y **H/M** son los que guían *[[ATAM]]* y *[[SAAM]]*. Es input directo del **paso 2 de ADD** ("identificar drivers") (source: arbol-de-utilidad.md).

CUÁNDO NO · CUIDADO

Anti-patrones del árbol: escenarios vagos sin métrica ("debe ser rápido"); priorización sesgada por un único stakeholder; árbol completo pero estático que no se revisa; **H/H por todas partes** — dilución de prioridades: el árbol debe ser discriminante (source: arbol-de-utilidad.md).

EN EL PARCIAL

Una hoja del árbol = un **escenario bien formado** de 6 partes: *source*, *stimulus*, *environment*, *artifact*, *response*, *response measure*. Sin **response measure** (la métrica verificable) no es un escenario, es una aspiración (source: arbol-de-utilidad.md).

§3 Trade-offs canónicos

FUENTE

"Ningún atributo mejora en el vacío." La cátedra (Clase 2) lista los conflictos clásicos: *Performance vs Maintainability · Availability vs Consistency · Security vs Usability* (source: [atributos-de-calidad.md](#); [teorema-cap.md](#)).

TABLA 2. Trade-offs canónicos atributo ↔ atributo

Tensión	Gana	Resigna	Mecanismo típico
Security ↔ Usability	SECURITY	Usability	MFA, timeouts, rotación de credenciales
Performance ↔ Maintainability	PERFORMANCE	Maintainability	cachés, denormalización, código vectorizado
Availability ↔ Consistency	AVAILABILITY	Consistency	teorema CAP (ver §4)

Otros ejes que menciona la cátedra: *Portability vs Performance* (capas de abstracción), *Reusability vs Simplicity* (generalización prematura), *Testability vs Deliverability* (source: [atributos-de-calidad.md](#)).

TRADE-OFF

Security ↔ Usability: MFA, timeouts y rotación agregan fricción al usuario. Se mitiga aplicándolos selectivamente por rol/riesgo, no a todos por igual (source: [atributos-de-calidad.md](#)).

TRADE-OFF

Performance ↔ Maintainability: cachés y denormalización aceleran, pero introducen estado duplicado y código más difícil de modificar. El costo se paga en cada cambio futuro del esquema (source: [atributos-de-calidad.md](#)).

TRADE-OFF

Availability ↔ Consistency: es exactamente **CAP**. Bajo partición, mantener todos los nodos respondiendo (A) implica aceptar lecturas obsoletas; garantizar lectura de la última escritura (C) implica rechazar peticiones. Detalle en §4 (source: [teorema-cap.md](#)).

§4 CAP como trade-off

DEFINICIÓN

Teorema CAP (Brewer 2000; prueba: Gilbert & Lynch 2002): en un sistema de datos distribuido, ante una **partición de red**, hay que elegir entre **Consistency** y **Availability** — no se pueden ofrecer ambas a la vez (source: [teorema-cap.md](#)).

C — Consistency

Toda lectura ve la última escritura confirmada (linealizabilidad). *"All clients see current data regardless of updates or deletes."*

A — Availability

Toda petición a un nodo sano recibe respuesta (no error, no timeout). *"The system continues to operate as expected even with node failures."*

P — Partition Tolerance

El sistema sigue operando aunque la red entre nodos se rompa.

REGLA DE ORO

"Elegí 2 de 3" es engañoso: P no es opcional — toda red real eventualmente particiona. La elección real es **C vs A durante una partición** (source: teorema-cap.md).

CP vs AP

CP — Consistencia + tolerancia a partición. Durante la partición, el lado "minoritario" **rechaza operaciones** para no divergir: pierdo disponibilidad. *Ej.: HBase, MongoDB con majority writes, etcd, Zookeeper.*

AP — Disponibilidad + tolerancia a partición. Todos los nodos **siguen aceptando operaciones**; al reunificarse hay que **resolver conflictos**: pierdo consistencia momentánea (eventual consistency, LWW, vector clocks, CRDTs). *Ej.: Cassandra, DynamoDB, CouchDB, Riak.*

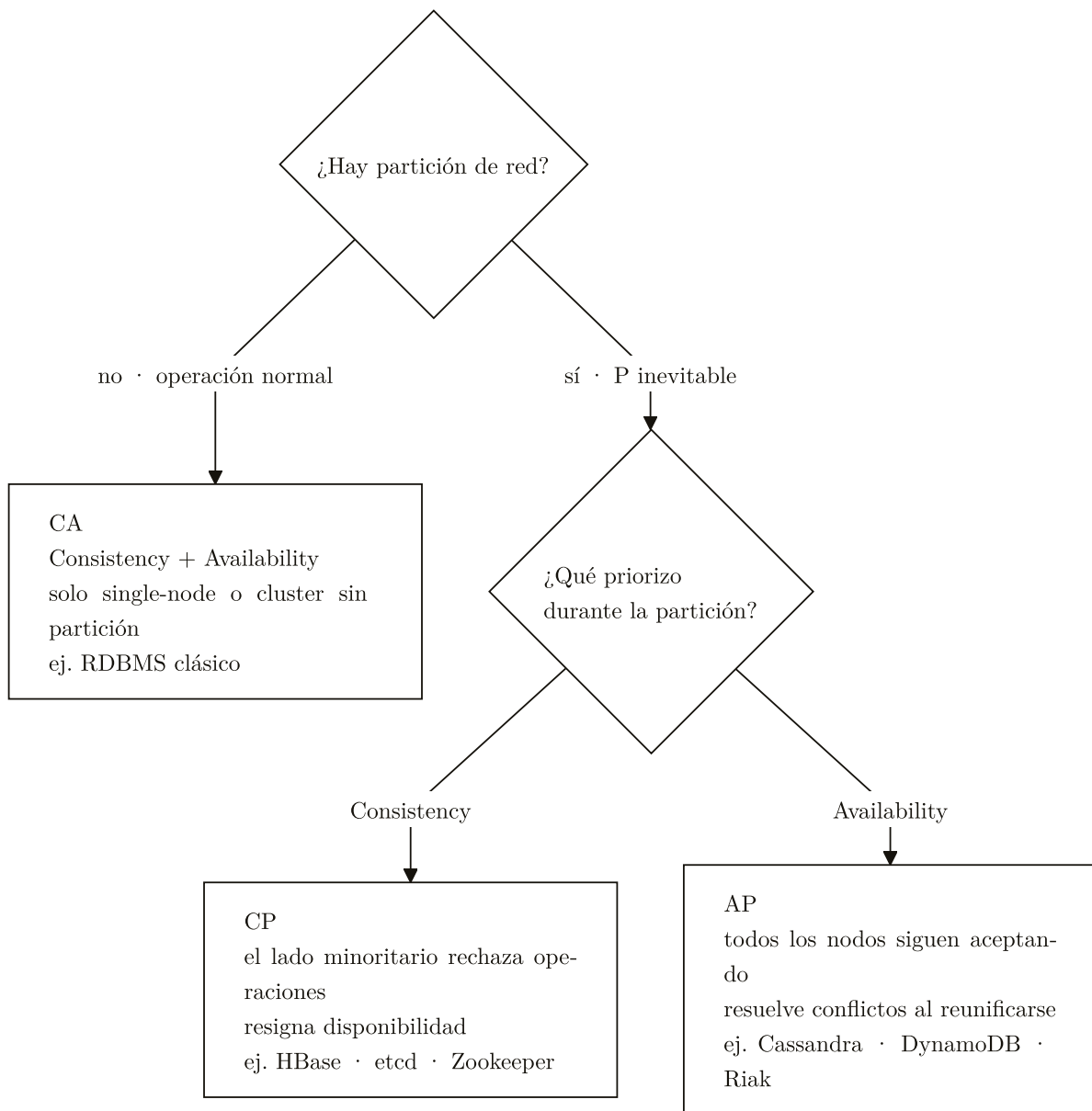


FIGURA 2. El espectro CAP como decisión: P es inevitable en redes reales, así que la elección real es C vs A durante la partición. CA solo existe cuando no hay partición.

CA sólo existe sin partición (single-node o cluster sin partición tolerable, ej. RDBMS clásico single-node). Asumir que P no ocurrirá es ingenuo (source: teorema-cap.md).

TABLA 3. Decisiones canónicas CP/AP

Caso	Elección	Por qué
Banca / transacciones financieras	CP	Consistencia no-negociable: prefiero rechazar a aceptar saldos divergentes.
Carrito de e-commerce	AP	Amazon: "siempre permitir agregar al carrito"; reconcilia después.
Feed de red social / cache de timelines	AP	Un seguidor menos por 30s no molesta; un 503 sí. Ver [[Caso Twitter — Big Data en tiempo real]].
Coordinación (locks, leader election)	CP	Zookeeper, etcd: la consistencia ES la razón de existir del componente.
Catálogo read-heavy	AP	Réplicas eventuales primary-secondary.

FUENTE

PACELC (Abadi 2010): durante partición (P) elegís A o C; **else** (E, operación normal) elegís Latency o Consistency. Cassandra es **PA/EL**; Spanner es **PC/EC** (CP siempre, a costa de latencia) (source: teorema-cap.md).

EN EL PARCIAL

Ningún ATAM serio sobre un sistema distribuido omite ubicar el sistema en el espectro **CP** ↔ **AP** y bajo qué escenarios opera de cada lado. Para responder bien: nombrá el atributo dominante (Consistency o Availability), elegí CP/AP con un ejemplo concreto, y explicitá el costo (qué se resigna durante la partición) (source: teorema-cap.md).